

Week 8

Versioning, Storing, and Documenting Data Workflows

Data versioning; documentation; storage; reproducible pipelines

- Certificate IV in Information Technology (Data Science and AI)
- Aligns with industry demand for reproducible and auditable AI workflows

Learning Objectives

- Understand the need for versioning and documenting data analysis steps in AI projects
- Gain practical experience with Git and DVC for tracking code, data, and results
- Learn best practices for storing and securing data in professional environments
- Develop workflow documentation skills for reproducibility and collaboration

Agenda

1. Why versioning and documentation matter in data science and AI
2. Tools overview; Git and DVC basics
3. Best practices for workflow documentation and storage
4. Industry case study; real-world workflow versioning
5. Hands-on activity; tracking analysis with Git and DVC
6. Assessment tips; connecting to your assignments

Why Versioning and Documentation?

- Data science teams work with constantly changing datasets and code; tracking changes prevents errors and loss
- Version control enables collaboration and auditability in research and commercial projects
- Documentation ensures that workflows are reproducible and transparent; critical for AI and compliance

Industry Standards and Context

- AI research labs require all analysis steps to be repeatable and traceable for trust and compliance
- Organizations working with large datasets (like LLM training) must log every change for data lineage
- Example; Dataset card standards and "datasheets for datasets" require clear version history and documentation

Introduction to Git

- Git is an industry-standard tool for versioning code and small text files (e.g., Jupyter Notebooks, scripts)
- Main concepts; repository, commit, branch, merge
- Enables easy tracking of analysis changes; allows rollback, collaboration, and review

```
# Example; Initializing a Git repository and tracking a notebook  
git init  
git add analysis.ipynb  
git commit -m "Initial version of data analysis notebook"
```

Introduction to DVC; Data Version Control

- DVC extends Git to handle large datasets and model files
- Tracks versions of datasets and links them to analysis code; supports reproducible pipelines
- Common in AI organizations where datasets are too large for Git alone

```
# Example; Versioning a dataset with DVC
dvc init
dvc add data/big_dataset.csv
git add data/big_dataset.csv.dvc .gitignore
git commit -m "Track dataset with DVC"
```

Best Practices for Versioning and Documentation

- Always keep key scripts, notebooks, and configs under version control
- Use descriptive commit messages and maintain an updated README with steps for reproduction
- Link code, data, and results with clear version references (e.g., Git SHA or DVC hash)
- Document data source, cleaning steps, and analysis logic

Secure and Organised Data Storage

- Store data and code in organized directories with clear naming schemes (e.g., /data, /notebooks, /results)
- Follow organizational policies for secure storage; use network drives or approved cloud solutions
- Avoid storing sensitive data in public repositories; follow institutional privacy guidelines

Case Study; OpenAI Dataset Versioning

- OpenAI and similar organizations use tools like Git and DVC to manage LLM datasets
- Every data processing step is versioned and documented; enables exact reproduction of published models
- Enables audits for data ethics, privacy, and compliance with AI standards

Hands-On Activity; Tracking a Data Analysis Project

- Create a Git repository for a new Jupyter Notebook analysis
- Use DVC to add and version a UCI dataset used in your EDA project
- Practice making commits whenever you change code, data, or results
- Write a short README to document each step and how to reproduce your workflow

Documenting Data Analysis Workflows

- Use clear README files to describe steps, data sources, required libraries, and instructions
- Jupyter Notebooks; add markdown cells to explain code, rationale, and results (each section should be clear to another reader)
- Metadata files (like dataset cards or config YAML) help document data and processing parameters

Connecting to Assessments

- Proper versioning, documentation, and storage support all assessment projects (notebooks, reports, applied ML tasks)
- Good workflow documentation helps you (and assessors) verify your work; critical for reassessment
- Ensure analysis steps, results, and data sources are reproducible and clearly explained in submissions

Summary and Next Steps

- Reproducibility and good documentation are essential in AI and data science careers
- Practice using Git and DVC to manage code and data in your current and future projects
- Next week; Data Policies, Compliance, and Organizational Best Practices

Additional Resources and Practice

- Git Documentation; <https://git-scm.com/docs/>
- DVC Tutorials; <https://dvc.org/doc/start>
- Example; "datasheets for datasets" standard; <https://arxiv.org/abs/1803.09010>

Practice until you are comfortable committing, documenting, and sharing your analysis workflows

